



PROJET DE FIN D'ETUDES

Rapport détaillé des travaux réalisés

TuniSign - Plateforme intelligente et accessible d'apprentissage de la langue des signes

Étudiant	À compléter
Encadrant	À compléter
Période	À compléter
Version	Septembre 2026

Prototype fonctionnel avancé : frontend gamifié, API sécurisée, reconnaissance gestuelle, progression, certificats et administration.

Sommaire

1. Résumé exécutif	4
2. Contexte et problématique	4
3. Objectifs définis	4
3.1 Objectifs fonctionnels	4
3.2 Objectifs techniques	5
4. Architecture mise en place	5
Technologies principales	6
5. Travaux réalisés sur le frontend	6
5.1 Refonte visuelle et identité TuniSign	6
5.2 Navigation publique et navigation authentifiée	6
5.3 Page d'accueil personnalisée	6
5.4 Catalogue pédagogique	7
5.5 Parcours d'apprentissage inspiré de Duolingo	7
5.6 Espace personnel et paramètres	7
5.7 Contrôle du mot de passe	7
5.8 Internationalisation	8
6. Travaux réalisés sur le backend Spring Boot	8
6.1 Modèle de données et API REST	8
6.2 Authentification classique par JWT	8
6.3 Vérification de l'adresse e-mail	8
6.4 Connexion et création de compte avec Google	9
6.5 Authentification à deux facteurs facultative	9
6.6 Administration	9
6.7 Gestion des erreurs	9
7. Travaux réalisés sur le service d'intelligence artificielle	10
7.1 Intégration d'un service FastAPI	10
7.2 Prétraitement des mains	10
7.3 Gestion des classes du modèle	10
7.4 Amélioration des confusions	10
7.5 Pipeline de données et entraînement	10
7.6 Point scientifique important	11

8. Badges, progression et certificats	11
8.1 Gamification	11
8.2 Condition du certificat	11
8.3 Certificat numérique vérifiable	11
9. Sécurité mise en œuvre	12
10. Tests et qualité	12
10.1 Frontend	12
10.2 End-to-end	12
10.3 Backend	13
10.4 Service IA	13
10.5 Intégration continue	13
11. Difficultés rencontrées et solutions apportées	14
12. État actuel et limites	14
Réalisé et démontrable	14
Fonctionnel sous configuration ou données supplémentaires	15
À finaliser avant production	15
13. Indicateurs du dépôt	15
14. Scénario conseillé pour la démonstration à l'encadrant	16
15. Plan de présentation orale en dix minutes	16
Minute 1 - Problème	16
Minutes 2-3 - Solution et architecture	16
Minutes 4-5 - Expérience utilisateur	16
Minutes 6-7 - Intelligence artificielle	16
Minute 8 - Sécurité	16
Minute 9 - Qualité	16
Minute 10 - Limites et suite	17
16. Conclusion	17

1. Résumé exécutif

TuniSign est une plateforme web d'apprentissage assisté de la langue des signes. Le projet reprend certains principes pédagogiques des plateformes gamifiées comme Duolingo : progression par leçons, observation d'un geste, reproduction devant une webcam, score, mini-tests, badges et certificat final.

Le travail réalisé couvre trois blocs techniques principaux :

1. une interface web React/TypeScript moderne, responsive, animée et multilingue ;
2. une API Spring Boot sécurisée, connectée à PostgreSQL, qui gère les comptes, les contenus pédagogiques, les rôles et les certificats ;
3. un microservice FastAPI/TensorFlow qui traite les images de la webcam et prépare la reconnaissance de lettres, chiffres et expressions.

Le projet est aujourd'hui un **prototype fonctionnel avancé**. Les parcours essentiels sont présents : inscription, validation de l'e-mail, connexion classique ou Google, 2FA facultative, espace personnel, catalogue, apprentissage par webcam, récompenses, certificat vérifiable et administration. Une base de tests automatisés et de protections inspirées de l'OWASP Top 10 a également été ajoutée.

Il ne faut cependant pas présenter la solution comme déjà prête pour une production publique. Certaines parties doivent encore être consolidées : stockage serveur de toute la progression, modèle réellement entraîné sur des données de langue des signes tunisienne, déploiement HTTPS, stockage du JWT en cookie sécurisé, supervision et test d'intrusion indépendant.

2. Contexte et problématique

L'apprentissage de la langue des signes est fortement visuel. Une plateforme classique qui affiche seulement du texte ou des images ne fournit pas suffisamment de retour à l'apprenant. L'objectif du projet est donc de proposer une expérience interactive dans laquelle l'utilisateur peut :

- découvrir un signe ou une lettre grâce à une référence visuelle ;
- observer une vidéo pédagogique avant de commencer un cours ;
- reproduire le geste devant sa webcam ;
- recevoir une prédiction et un score fournis par un modèle d'intelligence artificielle ;
- répéter les gestes difficiles et passer un test final ;
- suivre sa progression et recevoir des badges ;
- obtenir un certificat numérique vérifiable après avoir terminé un cours.

Le projet répond aussi à des besoins non fonctionnels importants : sécurité des comptes, confidentialité, gestion des rôles, multilinguisme, ergonomie et possibilité d'administrer le contenu sans modifier directement le code.

3. Objectifs définis

3.1 Objectifs fonctionnels

- Créer et authentifier un utilisateur.
- Vérifier la validité de son adresse e-mail.
- Autoriser une connexion avec Google.
- Proposer une authentification à deux facteurs facultative avec Google Authenticator.

- Afficher un espace personnel contenant les informations, statistiques et paramètres du compte.
- Organiser les apprentissages par catégories, cours et signes.
- Utiliser la webcam pour analyser automatiquement un geste.
- Gérer les échecs, les rappels pédagogiques, les aides et les mini-tests.
- Attribuer des badges et délivrer un certificat en fin de cours.
- Vérifier publiquement un certificat par code et QR code.
- Fournir un tableau de bord administrateur.
- Proposer l'interface en français, anglais et arabe tunisien.

3.2 Objectifs techniques

- Séparer le frontend, le backend métier et le service IA.
- Exposer une API REST documentée avec Swagger/OpenAPI.
- Utiliser PostgreSQL et JPA pour la persistance.
- Sécuriser les API avec Spring Security et JWT.
- Construire une chaîne de collecte, entraînement et évaluation des modèles IA.
- Ajouter des tests unitaires, d'intégration et end-to-end.
- Mettre en place une intégration continue avec GitHub Actions.
- Réduire les principaux risques décrits par l'OWASP Top 10.

4. Architecture mise en place



Technologies principales

Couche	Technologies utilisées
Frontend	React 19, TypeScript, Vite, CSS personnalisé
Backend	Java 21, Spring Boot 3.5, Spring Security, Spring Data JPA
Base de données	PostgreSQL, Hibernate/JPA
Authentification	JWT, BCrypt, Google Identity Services, TOTP/Google Authenticator
IA	Python 3.12, FastAPI, TensorFlow 2.18, Keras, MediaPipe, OpenCV, Pillow, NumPy
Certificats	signature numérique, QR code ZXing
Tests	Vitest, Testing Library, JUnit 5, Mockito, MockMvc, Playwright, <code>unittest</code>
Sécurité/qualité	npm audit, OWASP Dependency-Check, OWASP ZAP, GitHub Actions

5. Travaux réalisés sur le frontend

5.1 Refonte visuelle et identité TuniSign

- Remplacement de l'ancienne identité « Dexora » par **TuniSign**.
- Intégration du logo SINA/TuniSign dans l'en-tête et les pages principales.
- Mise à jour du titre et de l'icône de l'onglet du navigateur.
- Création d'une charte sombre/bleu pétrole avec turquoise, cartes arrondies, ombres et effets de profondeur.
- Ajout d'un mode clair/sombre.
- Création d'animations sur les boutons, cartes, transitions, célébrations et indicateurs de progression.
- Adaptation responsive pour ordinateur, tablette et mobile.
- Suppression de la bannière permanente « mode démonstration » qui nuisait à l'expérience.

5.2 Navigation publique et navigation authentifiée

- Avant authentification, seules les actions publiques sont affichées.
- Les menus privés « Apprendre », « Mon espace », « Récompenses » et « Administration » sont protégés.
- Une tentative d'accès à une page privée redirige vers la page de connexion.
- Après authentification, la navigation est personnalisée selon le rôle.
- Le menu Administration est visible uniquement pour un compte ayant le rôle **ADMIN**.

Cette correction a éliminé la confusion initiale où la barre de navigation privée était visible avant la connexion.

5.3 Page d'accueil personnalisée

- Présentation publique de la plateforme pour un visiteur.
- Message et appels à l'action adaptés à un membre connecté.
- Suppression de l'option « Créer un compte » après connexion.
- Mise en avant de la pratique webcam, du score et de la série quotidienne.
- Redirection rapide vers le catalogue ou l'espace personnel.

5.4 Catalogue pédagogique

- Filtres par catégories : Toutes, Salutations, Alphabet ASL et Chiffres.
- Cartes de cours contenant titre, description, difficulté, XP et nombre de signes.
- Verrouillage ou activation dynamique d'un cours selon les capacités annoncées par le service IA.
- Données de démonstration utilisées comme solution de secours lorsque le backend est indisponible.
- Chargement des catégories, cours et signes réels depuis l'API.

5.5 Parcours d'apprentissage inspiré de Duolingo

Le parcours Alphabet comprend les étapes suivantes :

1. regarder une vidéo éducative A-Z avant les exercices ;
2. observer le geste de référence d'une lettre ;
3. activer la webcam et reproduire le geste ;
4. analyser automatiquement plusieurs images pour obtenir une prédiction stable ;
5. afficher le score, la lettre prédite et la lettre attendue ;
6. continuer vers la lettre suivante ;
7. passer un mini-test puis un test final aléatoire.

Les règles pédagogiques ajoutées sont :

- analyse automatique lorsque la caméra détecte une pose stable ;
- bouton permettant de passer manuellement à la lettre suivante ;
- après six échecs, affichage d'un rappel du geste de référence ;
- répétition de ce cycle de rappel jusqu'à trois fois ;
- si l'échec continue, déplacement de la lettre difficile à la fin du parcours au lieu de revenir incorrectement à la lettre précédente ;
- rappel particulier pour les confusions A/S ;
- quatre aides disponibles par cours ; une aide valide le geste choisi et son badge ;
- traitement spécifique prévu pour J et Z, qui nécessitent une séquence de mouvement.

5.6 Espace personnel et paramètres

- Affichage du nom, prénom, e-mail, série, XP et état de sécurité du compte.
- Carte « Continuer » et challenge hebdomadaire.
- Bouton **Paramètres du compte**.
- Modification des informations personnelles autorisées.
- E-mail protégé et affiché comme vérifié.
- Choix entre trois avatars visuels : signer, étudiant et explorateur.
- Changement de mot de passe.
- Configuration facultative de Google Authenticator.
- Distinction entre compte local et compte Google.

5.7 Contrôle du mot de passe

Le formulaire de création et le formulaire de changement du mot de passe imposent :

- 8 à 128 caractères ;

- au moins une lettre majuscule ;
- au moins une lettre minuscule ;
- au moins un chiffre ;
- au moins un caractère spécial ;
- aucun espace.

La règle est appliquée dans le navigateur pour guider l'utilisateur et dans le backend pour empêcher un contournement par Swagger, Postman ou une requête manuelle.

5.8 Internationalisation

- Interface française.
- Traduction anglaise.
- Traduction en arabe tunisien.
- Sélecteur de langue intégré à l'en-tête.
- Traduction des parcours, notifications, paramètres, récompenses et messages de sécurité.

6. Travaux réalisés sur le backend Spring Boot

6.1 Modèle de données et API REST

Le backend contient actuellement :

- 15 entités JPA ;
- 16 contrôleurs REST ;
- environ 77 opérations exposées par annotations HTTP ;
- des couches DTO, mapper, repository et service.

Les principaux domaines sont : utilisateurs, catégories, cours, signes, parcours, progression, badges, certificats, tokens de validation d'e-mail et clés de signature.

6.2 Authentification classique par JWT

- Inscription avec nom, prénom, e-mail et mot de passe.
- Hachage du mot de passe avec BCrypt.
- Connexion avec e-mail et mot de passe.
- Création d'un JWT Tunisign après authentification.
- Filtre chargé de lire et valider le Bearer Token.
- Expiration configurable du JWT.
- Secret JWT Base64 d'au moins 256 bits.
- Secret explicite obligatoire en profil de production.
- Gestion propre des JWT mal formés ou expirés.

6.3 Vérification de l'adresse e-mail

- Création d'un compte local dans l'état non vérifié.
- Génération d'un token de validation à usage limité.
- Lien valable 30 minutes par défaut.

- Blocage de la connexion tant que l'adresse n'est pas confirmée.
- Possibilité de renvoyer un lien de vérification.
- Envoi SMTP configurable.
- Lien de développement désactivé par défaut pour éviter une fuite en production.

6.4 Connexion et création de compte avec Google

- Intégration de Google Identity Services dans React.
- Envoi du credential Google au backend.
- Vérification côté serveur de la signature, de l'audience, de l'émetteur et de l'expiration.
- Création ou récupération du compte local associé.
- Émission d'un JWT interne TuniSign après validation Google.
- Aucun secret Google n'est placé dans le frontend.

6.5 Authentification à deux facteurs facultative

- La 2FA est un choix de l'utilisateur, pas une obligation globale.
- L'utilisateur configure Google Authenticator depuis son espace personnel.
- Génération d'un secret TOTP.
- Validation par un code à six chiffres.
- Lors d'une connexion ultérieure, le mot de passe est vérifié en premier.
- Si la 2FA est activée, une seconde page demande le code Google Authenticator.
- Le champ 2FA n'est donc plus affiché inutilement dans le formulaire principal de connexion.

6.6 Administration

Un dashboard administrateur a été développé avec :

- statistiques sur les utilisateurs, comptes actifs, e-mails vérifiés et 2FA ;
- statistiques sur les catégories, cours, signes, badges et certificats ;
- recherche et consultation des utilisateurs ;
- modification des rôles ;
- activation et désactivation des comptes ;
- création, modification et suppression des catégories ;
- création, modification et suppression des cours ;
- création, modification et suppression des signes.

Des protections supplémentaires empêchent :

- un administrateur de désactiver son propre compte ;
- la suppression ou rétrogradation du dernier administrateur ;
- un étudiant d'appeler directement les routes administrateur.

6.7 Gestion des erreurs

- Centralisation des exceptions avec `GlobalExceptionHandler`.
- Réponses structurées au format `ProblemDetail`.
- Erreurs de validation retournées champ par champ.
- Aucun détail de pile Java ou message SQL dans une réponse 500.

- Génération d'un identifiant d'incident pour retrouver l'erreur dans les logs.
- Codes cohérents : 400, 401, 403, 404, 409, 413, 415, 422, 429, 500 et 503.

7. Travaux réalisés sur le service d'intelligence artificielle

7.1 Intégration d'un service FastAPI

- Endpoint `/health` qui décrit l'état du modèle et ses capacités.
- Endpoint `/predict` pour une image statique.
- Endpoint `/predict-sequence` pour une série d'images représentant un geste dynamique.
- Communication directe entre le frontend et le service IA.
- Limitation à 5 Mo par image et 24 Mo par séquence.
- Validation des types JPEG, PNG et WebP.
- Configuration CORS explicite.

7.2 Prétraitement des mains

- Détection de la main avec MediaPipe Hands.
- Recadrage sur la zone de la main au lieu d'analyser tout l'arrière-plan.
- Normalisation de l'image à la taille attendue par le modèle.
- Test de l'image originale et de sa version miroir.
- Sélection de l'orientation qui fournit la meilleure confiance.
- Extraction et normalisation des 21 points de la main, soit 63 coordonnées.

7.3 Gestion des classes du modèle

- Vérification de l'ordre des classes grâce à des fichiers `classes.json`, `labels.json` et `metadata.json`.
- Détection d'un nombre de sorties incompatible avec le nombre de classes.
- Profils d'entraînement disponibles : Alphabet, Chiffres, Étendu, ancien profil 29 classes et classes disponibles.
- Préparation des classes A-Z, 0-9 et des expressions : BONJOUR, HI, THANK_YOU, I_LOVE_YOU et MERCI.

7.4 Amélioration des confusions

- Création d'un classifieur complémentaire basé sur les landmarks pour A/S.
- Normalisation gauche/droite des coordonnées.
- Renforcement prévu des classes difficiles A/S et M/N/T pendant l'entraînement.
- Déclaration de J et Z comme classes dynamiques, car une image fixe ne suffit pas à reconnaître leur trajectoire.

7.5 Pipeline de données et entraînement

Des scripts ont été développés pour :

- collecter des images depuis la webcam ;
- collecter des séquences vidéo ;
- créer un jeu de test indépendant ;
- entraîner le modèle statique ;

- entraîner le correcteur A/S ;
- entraîner un modèle GRU pour les expressions vidéo ;
- évaluer le modèle ;
- générer des rapports JSON/CSV ;
- auditer l'ordre des classes et les métadonnées.

L'augmentation de données prévoit rotations, translations, zoom, contraste, miroir, différents arrière-plans et éclairages.

7.6 Point scientifique important

Le modèle actuel et les références pédagogiques principales sont basés sur l'**ASL** pour l'alphabet et les chiffres. Le produit est présenté comme une plateforme de langue des signes tunisienne. Pour que cette affirmation soit scientifiquement correcte, il faudra constituer ou obtenir un jeu de données validé de langue des signes tunisienne, faire valider les gestes par des experts/signeurs tunisiens, réentraîner les modèles et mesurer leurs performances sur des utilisateurs indépendants.

Cette limite doit être présentée clairement à l'encadrant : l'architecture est prête pour la LST, mais le contenu IA actuel reste en partie un prototype ASL.

8. Badges, progression et certificats

8.1 Gamification

- XP par leçon.
- Série de jours.
- Progression visuelle par cours.
- Badge associé à chaque leçon/signe.
- Quatre aides disponibles par cours.
- Page Récompenses regroupant badges et certificats.
- Challenge hebdomadaire.

8.2 Condition du certificat

Le principe défini est le suivant : un cours contient plusieurs leçons, chaque leçon permet d'obtenir un badge, et le certificat devient disponible lorsque tous les badges du cours sont obtenus.

8.3 Certificat numérique vérifiable

- Nom de l'utilisateur et titre du cours inscrits sur le certificat.
- Date de délivrance.
- Code de vérification unique.
- Signature numérique du contenu canonique du certificat.
- Empreinte de la clé publique.
- QR code pointant vers une page publique de vérification.
- Détection d'un certificat révoqué ou dont la signature est invalide.
- Possibilité d'impression du certificat.

9. Sécurité mise en œuvre

Le projet inclut un premier durcissement aligné sur l'OWASP Top 10:2025 :

- contrôle d'accès par rôles et protection des routes sensibles ;
- réponses 401 et 403 homogènes ;
- limitation des tentatives sur connexion, inscription, Google et 2FA ;
- validation stricte des DTO ;
- politique forte pour les mots de passe ;
- BCrypt avec coût 12 ;
- secret JWT non codé en dur ;
- profil de production séparé ;
- Swagger désactivable et privé en production ;
- CORS limité aux origines configurées ;
- en-têtes CSP, anti-iframe, referrer policy et permissions policy ;
- limites de taille pour les requêtes et images ;
- erreurs internes sans fuite d'informations ;
- journalisation des échecs d'authentification et des actions administrateur ;
- audit npm et profil OWASP Dependency-Check ;
- exemple de commande OWASP ZAP Baseline.

Le détail du mapping OWASP et les commandes se trouvent dans [docs/SECURITY_TESTING.md](#).

10. Tests et qualité

10.1 Frontend

- Tests Vitest et Testing Library sur la frontière d'authentification.
- Vérification que les menus privés ne sont pas affichés à un visiteur.
- Vérification de la redirection vers la connexion.
- Vérification du chargement du compte authentifié.
- Tests du multilinguisme.
- Tests du traitement des erreurs API.
- Sept tests frontend validés lors de la dernière exécution.
- Lint TypeScript et build de production validés.

10.2 End-to-end

Playwright et Chromium testent le comportement réel dans un navigateur :

- page publique et titre TuniSign ;
- absence des menus privés avant connexion ;
- redirection d'un visiteur vers l'authentification ;
- contrôle d'un e-mail invalide ;
- refus d'un mot de passe faible et acceptation de `TuniSign1!`.

Résultat observé : **4 scénarios sur 4 réussis.**

10.3 Backend

- Validation des requêtes d'authentification.
- Validation des codes 2FA.
- Tests de création et de validation JWT.
- Rejet d'un token falsifié.
- Tests du filtre anti-bruteforce.
- Tests 401/403 des routes administrateur.
- Tests de protection du dernier administrateur.
- Test de non-divulgateur d'une erreur interne.
- Tests de la politique de mot de passe.

La suite backend complète avait validé 14 tests. Les deux tests ciblés de la nouvelle politique de mot de passe ont ensuite été exécutés avec succès. Dans l'environnement Codex Windows, Maven rencontre parfois un verrouillage [ZipFS](#) lors de la fermeture d'un JAR local ; il s'agit d'une contrainte du cache/sandbox et non d'un échec fonctionnel du test ciblé.

10.4 Service IA

- Vérification de la continuité des indices de classes.
- Correspondance entre labels et indices.
- Normalisation des landmarks.
- Réflexion d'une main gauche vers une orientation canonique.
- Vérification des profils Alphabet, Chiffres, Étendu et ancien modèle.
- Vérification des labels de séquences et du rééchantillonnage.

Résultat observé : **8 tests IA sur 8 réussis**.

10.5 Intégration continue

Un workflow GitHub Actions exécute :

- installation reproductible des dépendances ;
 - lint et tests frontend ;
 - build Vite ;
 - audit npm ;
 - installation Chromium et tests Playwright ;
 - tests Maven du backend ;
 - tests Python du service IA.
-

11. Difficultés rencontrées et solutions apportées

Difficulté	Analyse	Solution mise en œuvre
Erreurs 403 après connexion	Token mal copié ou header Bearer incorrect	Documentation des étapes Swagger et contrôle JWT
Token masqué par *****	Copie depuis la fenêtre Authorize au lieu de la réponse brute	Utilisation du token complet de <code>/auth/login</code>
Difficulté <code>DEBUTANT</code> rejetée	Valeur absente de l'énumération Java	Utilisation des valeurs compatibles et correction du contenu
Champ 2FA affiché trop tôt	2FA facultative mélangée à la première étape	Séparation mot de passe puis page TOTP
Frontend identique avant/après connexion	État d'authentification insuffisamment pris en compte	Accueil et navigation conditionnels
Webcam vide	Flux vidéo attaché avant le rendu de l'élément	Attachement après montage et gestion des erreurs caméra
Modèle peu fiable	Fond complet, orientation et ordre des classes incertains	MediaPipe, recadrage, miroir, métadonnées et pipeline de collecte
Confusion A/S	Signes très proches au niveau de la forme	Correcteur par landmarks et conseil pédagogique sur le pouce
Retour erroné à la lettre précédente	Mauvaise logique après plusieurs échecs	Passage à la suivante et report de la difficile à la fin
TensorFlow 2.18 introuvable	Version de Python incompatible	Utilisation de Python 3.12 et environnement virtuel dédié
Réponses 422 FastAPI	Format multipart/champ incorrect	Standardisation de <code>image</code> et <code>images</code> dans le frontend
Maven sous Windows	Verrouillage ponctuel d'archives du cache	Dépôt Maven local au projet et exécution ciblée des tests

12. État actuel et limites

Réalisé et démontrable

- Interface TuniSign complète et responsive.
- Authentification locale et JWT.
- Validation e-mail sous configuration SMTP.
- Connexion Google sous configuration d'un Client ID.
- 2FA facultative avec Google Authenticator.
- Espace utilisateur et choix d'avatar.
- Catalogue et parcours Alphabet.
- Webcam et communication avec FastAPI.
- Pipeline IA de collecte/entraînement/évaluation.
- Badges et certificat signé avec QR code.
- Dashboard administrateur.
- Trois langues.

- Tests automatisés et CI.
- Premier durcissement OWASP.

Fonctionnel sous configuration ou données supplémentaires

- Envoi réel des e-mails : nécessite un compte SMTP et ses secrets.
- Connexion Google : nécessite la configuration Google Cloud.
- Reconnaissance des chiffres : nécessite que le modèle étendu 0-9 entraîné soit disponible.
- Expressions vidéo : nécessite un nombre suffisant de séquences par classe et le modèle GRU généré.
- Qualité de reconnaissance : dépend directement de la quantité, diversité et qualité du jeu de données.

À finaliser avant production

1. Enregistrer toute la progression et les badges côté serveur.
2. Ne jamais faire confiance au nombre de badges envoyé par le navigateur lors de l'émission d'un certificat.
3. Migrer le JWT de `localStorage` vers un cookie `HttpOnly`, `Secure`, `SameSite` avec protection CSRF adaptée.
4. Utiliser Redis ou une passerelle pour la limitation distribuée des tentatives.
5. Déployer derrière HTTPS avec HSTS.
6. Gérer les secrets dans un coffre ou dans les variables sécurisées de l'hébergeur.
7. Héberger/compresser la vidéo de 27 Mo et optimiser le logo d'environ 1,8 Mo.
8. Construire et valider un vrai corpus de langue des signes tunisienne.
9. Mesurer précision, rappel, F1-score et matrice de confusion sur un jeu indépendant.
10. Ajouter observabilité, alertes, sauvegardes PostgreSQL et politique de restauration.
11. Effectuer une revue OWASP ASVS et un test d'intrusion indépendant.
12. Déployer les trois services et documenter la procédure de livraison.

13. Indicateurs du dépôt

À la date du rapport :

- 47 commits dans la branche principale ;
- 135 fichiers Java de production ;
- 7 classes de tests Java ;
- 15 entités JPA ;
- 16 contrôleurs ;
- environ 77 opérations REST annotées ;
- 12 fichiers Python pour le service IA, ses scripts et ses tests ;
- 14 fichiers TypeScript/TSX principaux et E2E hors dépendances.

Attention Git : les dernières évolutions de sécurité, d'administration et de tests sont encore présentes comme modifications locales non validées dans l'état observé. Elles doivent être relues, ajoutées, commitées puis poussées sur `main` avant de considérer le dépôt distant comme à jour.

14. Scénario conseillé pour la démonstration à l'encadrant

1. Présenter l'accueil public, le logo, le thème et les trois langues.
2. Créer un compte et montrer le refus d'un mot de passe faible.
3. Montrer la page de vérification de l'e-mail.
4. Se connecter puis expliquer la seconde étape 2FA facultative.
5. Ouvrir Mon espace et modifier l'avatar.
6. Ouvrir le catalogue et sélectionner Alphabet A-Z.
7. Montrer la vidéo pédagogique obligatoire.
8. Observer une lettre puis activer la webcam.
9. Expliquer la prédiction IA, le score, le miroir et MediaPipe.
10. Montrer la règle des six échecs, du rappel et du report en fin de parcours.
11. Présenter les quatre aides, les badges et le test final.
12. Ouvrir le certificat, scanner le QR code et afficher sa page de vérification.
13. Se connecter comme administrateur et montrer utilisateurs, statistiques et contenu.
14. Terminer par les tests automatisés et les limites scientifiques du modèle actuel.

15. Plan de présentation orale en dix minutes

Minute 1 - Problème

« L'apprentissage de la langue des signes nécessite un retour visuel et pratique. Une simple bibliothèque de vidéos ne mesure pas si l'utilisateur reproduit correctement le geste. »

Minutes 2-3 - Solution et architecture

Présenter React, Spring Boot/PostgreSQL et FastAPI/TensorFlow, puis expliquer leur séparation.

Minutes 4-5 - Expérience utilisateur

Montrer inscription, espace personnel, catalogue, vidéo, webcam, score, test et récompenses.

Minutes 6-7 - Intelligence artificielle

Expliquer MediaPipe, recadrage, miroir, ordre des classes, modèle statique, séquences et confusions A/S.

Minute 8 - Sécurité

Présenter JWT, BCrypt, e-mail vérifié, Google, 2FA, rôles, anti-bruteforce et protections OWASP.

Minute 9 - Qualité

Présenter les tests unitaires, backend, IA, Playwright et GitHub Actions.

Minute 10 - Limites et suite

Expliquer honnêtement que l'architecture est avancée mais qu'un corpus LST validé, un stockage serveur complet de la progression et un déploiement sécurisé restent nécessaires.

16. Conclusion

Le projet a évolué d'une API CRUD et d'une interface simple vers une plateforme complète d'apprentissage assisté par IA. Le travail ne s'est pas limité au design : il a couvert l'architecture, la sécurité, l'authentification, la pédagogie, la reconnaissance visuelle, les récompenses, l'administration et la qualité logicielle.

La principale valeur technique est l'intégration cohérente de trois systèmes différents : une expérience React gamifiée, une API Spring Boot sécurisée et un pipeline IA FastAPI/TensorFlow. La prochaine étape prioritaire consiste à transformer ce prototype avancé en produit scientifiquement valide pour la langue des signes tunisienne, avec un jeu de données fiable, une progression entièrement autoritaire côté serveur et une infrastructure de production sécurisée.